

Universidad Andrés Bello
Facultad de Ingeniería
Carrera: Ingeniería en Computación e Informática
Asignatura: Paradigmas de Programación

Tarea 2

Sistema de gestión de rutinas de entrenamiento personalizadas

Integrante:	Mary González – 27.638.057-5
NRC:	8025
Ciudad:	Viña del Mar
Fecha de entrega:	Jueves 28 de mayo de 2026

1. Introducción

El presente informe describe el diseño e implementación de un sistema de gestión de rutinas de entrenamiento personalizadas desarrollado en Java con interfaz gráfica Swing. La solución permite cargar ejercicios desde una base de datos SQLite, mantenerlos en memoria durante la ejecución y generar una rutina según el tipo de ejercicio, nivel de intensidad y semana de trabajo del cliente.

La motivación del sistema es apoyar el trabajo de instructores deportivos que deben construir rutinas considerando objetivos físicos, niveles de exigencia y restricciones de no repetición en semanas consecutivas. El documento se organiza en una descripción general de la solución, el modelo de clases, las decisiones de diseño y una conclusión sobre el trabajo realizado.

2. Descripción de la solución

La solución se estructuró tomando como referencia el ejemplo Pokémon desarrollado en clases. En dicho ejemplo, una clase general Pokemon es especializada por clases hijas como PokemonAgua o PokemonTierra, mientras una clase Gimnasio administra registros y operaciones. En esta tarea, esa misma lógica se adaptó al dominio de rutinas deportivas: Ejercicio funciona como clase base, EjercicioCardiovascular y EjercicioFuerza representan especializaciones, y RutinaService cumple el rol de backend principal.

Por decisión de implementación, las funcionalidades de carga, generación, revisión y resumen fueron integradas en una única ventana principal. Esta ventana contiene botones y eventos que permiten avanzar por las distintas etapas del flujo. Aunque el enunciado propone ventanas separadas, la solución mantiene la separación lógica mediante los paquetes backend y frontend, y concentra la interacción en una interfaz única para simplificar la ejecución.

2.1 Clases principales

Clase	Paquete	Responsabilidad
Ejercicio	backend	Representa un ejercicio con código, nombre, tipo, intensidad, tiempo, descripción y última semana de uso.
GestorEjerciciosBD	backend	Gestiona la conexión con SQLite, crea la tabla, inserta ejercicios y carga ejercicios desde la base de datos.
RutinaService	backend	Administra la lógica del sistema: carga ejercicios,

		guarda datos en SQLite, genera rutinas, mantiene la rutina en memoria y calcula resúmenes.
VentanaPrincipal	frontend	Interfaz gráfica principal. Permite cargar ejercicios, generar rutina, revisar ejercicio por ejercicio y mostrar resumen final.
Main	Raíz del proyecto	Punto de entrada de la aplicación. Crea el backend y abre la ventana principal.
PruebaSQLite	Raíz del proyecto	Clase de prueba para verificar conexión con SQLite. Puede mantenerse como apoyo, pero no es parte del flujo principal.

2.2 Funcionamiento general

Al iniciar el sistema, la ventana principal solicita cargar una base de datos SQLite. La base debe contener una tabla llamada ejercicios con los campos solicitados en el enunciado.

Una vez cargados los ejercicios, el backend los almacena en una estructura ArrayList, equivalente al vector solicitado, y calcula estadísticas: cantidad total, tiempo total disponible, cantidad por tipo y cantidad por nivel.

Para generar la rutina, el usuario ingresa cliente, objetivo físico, semana actual, cantidad de ejercicios cardiovasculares, nivel cardiovascular, cantidad de ejercicios de fuerza y nivel de fuerza.

El backend selecciona ejercicios disponibles que coincidan con tipo y nivel, y descarta ejercicios cuyo campo semana_ultimo_uso sea igual a semana_actual - 1. Con esto se cumple la restricción de no repetir ejercicios en semanas consecutivas.

La revisión de rutina conserva el estado de navegación mediante una posición actual. Cuando la posición es la primera, el botón Volver queda deshabilitado. Cuando la posición corresponde al último ejercicio, el botón Siguiente cambia a Resumen de la rutina.

2.3 Diagrama de clases

El siguiente diagrama resume las relaciones principales entre las clases diseñadas:

Diagrama de clases - Sistema de gestión de rutinas de entrenamiento

Implementación real del proyecto: Java Swing + SQLite

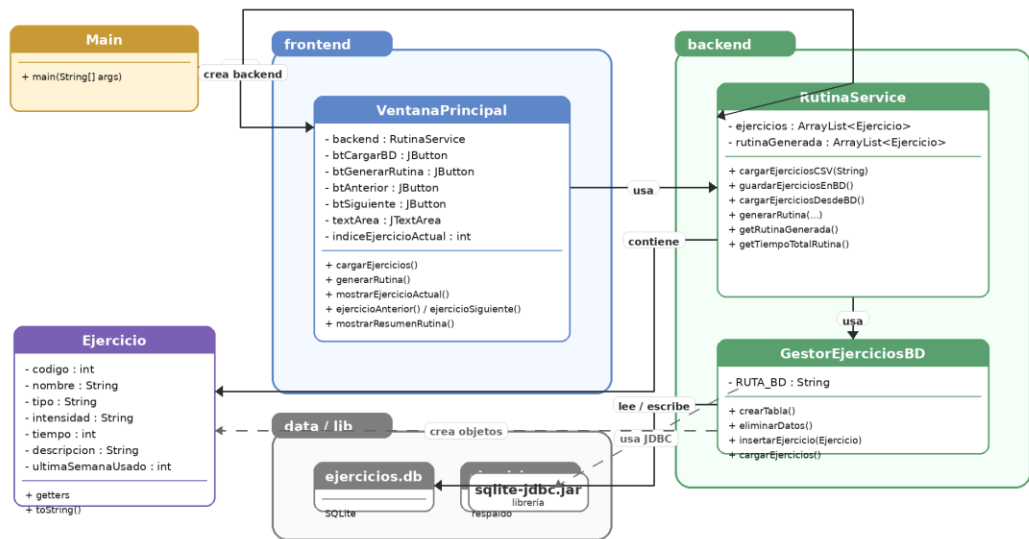


Figura 1. Diagrama de clases del sistema. Main crea el backend y abre VentanaPrincipal. VentanaPrincipal usa RutinaService para cargar, generar, revisar y resumir la rutina. RutinaService administra listas d

2.4 Decisiones de diseño

- Se utilizó una clase Ejercicio como modelo principal para representar los datos de cada ejercicio.
- Se separó la lógica en backend y la interfaz en frontend.
- Se utilizó ArrayList<Ejercicio> para mantener los ejercicios cargados en memoria.
- Se utilizó SQLite como base de datos local.
- Se creó GestorEjerciciosBD para separar el acceso a datos de la lógica principal.
- Se utilizó Swing con JFrame, JPanel, JButton, JTextArea, JOptionPane y ActionListener.
- Se manejaron errores con try/catch, especialmente para problemas de conexión, carga de datos y conversión numérica.

2.5 Cumplimiento de requerimientos

Requerimiento	Implementación
Ventana principal	VentanaPrincipal, desarrollada con Java Swing.
Carga desde BD	Botón “Cargar ejercicios desde SQLite”, usando GestorEjerciciosBD.
Generación de rutina	Botón “Cargar ejercicios desde SQLite”, usando GestorEjerciciosBD.
Revisión de rutina	Se muestra un ejercicio a la vez dentro de VentanaPrincipal.
Navegación	Botones “Anterior” y “Siguiente”

Resumen de rutina	Se muestra al llegar al último ejercicio mediante el botón “Resumen de la rutina”
Persistencia	Base de datos SQLite en data/ejercicios.db.
Archivo de respaldo	CSV en data/ejercicios.csv.
Mantenición en memoria	Uso de ArrayList<Ejercicio> y ArrayList<Ejercicio> rutinaGenerada.
Validación de errores	Manejo con try/catch, Exception, NumberFormatException y mensajes con JOptionPane.
Separación backend/frontend	Paquetes backend y frontend.
Programación orientada a objetos	Uso de addActionListener en los botones de la interfaz.

3. Conclusión

El sistema desarrollado cumple con los objetivos propuestos, ya que permite cargar ejercicios desde una base de datos, visualizar estadísticas iniciales, generar rutinas personalizadas y revisar los ejercicios seleccionados de forma ordenada. La solución integra programación orientada a objetos, herencia, modularización, eventos de interfaz gráfica y persistencia de información.

El trabajo permitió aplicar los conceptos vistos en clases en un caso distinto al ejemplo Pokemon, manteniendo una estructura similar pero adaptada a un problema real de gestión de rutinas deportivas. Además, el uso de SQLite permitió incorporar una fuente de datos externa y validar condiciones relevantes.